

GUÍA DE LABORATORIO PARCIAL

Arquitectura de Odoo 17 -19 y Framework OWL

Sistemas ERP y Desarrollo de Módulos

Asignatura:	Ebusiness y su contexto
Programa:	Ingeniería de Sistemas
Estudiantes:	María Isabel Mayorgano Arrieta, Maira Alejandra Narváez Ruz
Duración:	1.5 horas (+ autónomo)
Fecha:	/ /2026

1. Objetivos de Aprendizaje

1.1 Objetivo General

Comprender la arquitectura de Odoo 17 – 19 como sistema ERP modular y desarrollar competencias básicas en el framework OWL (Odoo Web Library) para la creación de componentes frontend reactivos.

1.2 Objetivos Específicos

- Identificar los componentes principales de la arquitectura de tres capas de Odoo
- Analizar la estructura de un módulo Odoo y sus archivos fundamentales

- Comprender el patrón de diseño MVC implementado en Odoo
- Crear componentes OWL básicos con estado reactivo y eventos
- Integrar componentes OWL en vistas de Odoo

2. Marco Teórico

2.1 Arquitectura de Odoo

Odoo implementa una arquitectura de tres capas bien definida que separa las responsabilidades del sistema:

CAPA	TECNOLOGÍA	FUNCIÓN
Presentación	OWL Framework, JavaScript, XML (QWeb), CSS/SCSS	Interfaz de usuario, componentes reactivos, renderizado de vistas
Lógica de Negocio	Python 3.10+, Werkzeug, API Odoo	Modelos, controladores, validaciones, workflows, reportes
Datos	PostgreSQL, ORM Odoo	Persistencia, relaciones, consultas, índices, seguridad de datos

2.2 Estructura de un Módulo Odoo

Un módulo Odoo es una unidad autocontenida de funcionalidad. Su estructura típica incluye:

```

mi_modulo/
├── __init__.py           # Inicialización del módulo Python
├── __manifest__.py       # Metadatos y dependencias del módulo
├── models/               # Definición de modelos (lógica de negocio)
│   ├── __init__.py
│   └── mi_modelo.py
├── views/                # Vistas XML (interfaces de usuario)
│   └── mi_modelo_views.xml
├── security/              # Control de acceso
│   └── ir.model.access.csv
├── static/                # Recursos estáticos
│   └── src/
│       ├── js/           # Componentes OWL
│       ├── xml/          # Plantillas QWeb
│       └── scss/          # Estilos
└── data/                 # Datos de demostración

#En esta arquitectura no se representa los Epecs que deben subirse como el laboratorio anterior

```

2.3 OWL Framework

OWL (Odoo Web Library) es el framework JavaScript moderno de Odoo, introducido en la versión 14 y consolidado en las versiones 16 y 17 + . Sus características principales son:

- Sistema de componentes reactivo inspirado en Vue.js y React
- Plantillas XML (QWeb) compiladas en tiempo de ejecución
- Sistema de hooks similar a React (useState, useEffect, useRef)
- Virtual DOM para renderizado eficiente
- Integración nativa con servicios de Odoo (RPC, notificaciones, modales)

3. Prerrequisitos

3.1 Conocimientos Previos

- Fundamentos de programación orientada a objetos
- Conceptos básicos de Python
- Fundamentos de JavaScript ES6+
- Conocimiento básico de XML
- Comandos básicos de Docker

3.2 Entorno de Trabajo

Verificar que Odoo está corriendo correctamente en Docker:

```
# Verificar contenedores activos
docker ps

# Verificar logs de Odoo
docker logs <nombre_contenedor_odoo>

# Acceder al contenedor
docker exec -it <nombre_contenedor_odoo> bash
```

4. Actividades del Laboratorio

4.1 Actividad 1: Exploración de la Arquitectura

□ **Tiempo estimado: 30 minutos**

Objetivo: Identificar los componentes de la arquitectura de Odoo en el entorno Docker.

Procedimiento:

1. Listar los contenedores y sus conexiones de red:

```
docker network ls
```

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
7a07ba1b1c1a	bridge	bridge	local
0b189c9227a0	host	host	local
1f5101dd02b3	kind	bridge	local
a9e134d6658e	lab-ecommerce_default	bridge	local
bbeef2a64d0a	none	null	local
d07e5885aa4f	odoo-docker_default	bridge	local

```
docker network inspect <red_odoo>
```

```
C:\Windows\System32\cmd.exe x + v
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker network inspect odoo-docker_default
[
  {
    "Name": "odoo-docker_default",
    "Id": "d07e5885aa4fd0eafac1737206b24774ac441a952dc54701b0b89b9f2cf2cb4a",
    "Created": "2026-09-11T19:31:00.290257285Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {
      "com.docker.compose.config-hash": "b8202a1f875f7e933f8e5a69de3ecc2467875070663326ef9971046bd949e20b",
      "com.docker.compose.network": "default",
      "com.docker.compose.project": "odoo-docker",
      "com.docker.compose.version": "5.3.1"
    },
    "Containers": {
      "3c99242b9ac1185b1861494d413db05ce11716ef86560b257e2cb2c5e31c97b1": {
        "Name": "odoo-docker-db-1",
        "EndpointID": "02bef16cbca265bf0a5f11ece9d141e94f8e09b9ce840a46e82a1db10c78361b",
        "MacAddress": "a6:19:68:e2:48:28",
        "IPv4Address": "172.20.0.2/16",
        "IPv6Address": ""
      },
      "cfc33f28b58f5ee80cbb8ec1feb66df7461b66bfadc67ba8dbc44922387aca": {
        "Name": "odoo-docker-odoo-1",
        "EndpointID": "476bdb173cc7838eb581d6944a6fa8a98a43e0568cfe04099d57ebd5c64106a3",
        "MacAddress": "f6:d3:b8:8a:81:82",
        "IPv4Address": "172.20.0.3/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          {
            "172.20.0.0/16": {
              "IPsInUse": 5,
              "DynamicIPsAvailable": 65531
            }
          }
        ]
      }
    }
  }
]
```

Se identificó la red Docker Compose odoo-docker_default, configurada con el controlador bridge, la subred 172.20.0.0/16 y el gateway 172.20.0.1. En esta red se encuentran conectados los contenedores odoo-docker-db-1, correspondiente a PostgreSQL, con IP 172.20.0.2, y odoo-docker-odoo-1, correspondiente a Odoo, con IP 172.20.0.3. Esto permite la comunicación entre Odoo y PostgreSQL dentro de la misma red Docker.

2. Explorar la estructura de directorios de Odoo:

```
docker exec -it <contenedor_odoo> ls -la /usr/lib/python3/dist-packages/odoo/
```

```
C:\Users\mayor\OneDrive\Docudocker exec odoo-docker-odoo-1 ls -la /usr/lib/python3/dist-packages/odoo/
total 844
drwxr-xr-x 12 root root 4096 Sep  9 02:38 .
drwxr-xr-x  1 root root 4096 Sep  9 02:38 ..
-rw-r--r--  1 root root 4950 Sep  8 01:03 __init__.py
-rw-r--r--  1 root root  38 Sep  8 01:03 __main__.py
drwxr-xr-x  2 root root 4096 Sep  9 02:38 __pycache__
drwxr-xr-x 644 root root 24576 Sep  9 02:38 addons
-rw-r--r--  1 root root 56787 Sep  8 01:03 api.py
drwxr-xr-x  4 root root 4096 Sep  9 02:38 cli
drwxr-xr-x  3 root root 4096 Sep  9 02:38 conf
-rw-r--r--  1 root root 2954 Sep  8 01:03 exceptions.py
-rw-r--r--  1 root root 229336 Sep  8 01:03 fields.py
-rw-r--r--  1 root root 91992 Sep  8 01:03 http.py
-rw-r--r--  1 root root 11407 Sep  8 01:03 import_xml.rng
-rw-r--r--  1 root root 3672 Sep  8 01:03 logging.py
-rw-r--r--  1 root root 3663 Sep  8 01:03 loglevels.py
-rw-r--r--  1 root root 322124 Sep  8 01:03 models.py
drwxr-xr-x  3 root root 4096 Sep  9 02:38 modules
-rw-r--r--  1 root root 16176 Sep  8 01:03 netsvc.py
drwxr-xr-x  3 root root 4096 Sep  9 02:38 osv
-rw-r--r--  1 root root 1880 Sep  8 01:03 release.py
drwxr-xr-x  3 root root 4096 Sep  9 02:38 service
-rw-r--r--  1 root root 31504 Sep  8 01:03 sql_db.py
drwxr-xr-x  3 root root 4096 Sep  9 02:38 tests
drwxr-xr-x  7 root root 4096 Sep  9 02:38 tools
drwxr-xr-x  3 root root 4096 Sep  9 02:38 upgrade
```

```
docker exec -it <contenedor_odoo> ls -la /usr/lib/python3/dist-packages/odoo/addons/
```

```
C:\Windows\System32\cmd.exe X + v
C:\Users\mayor\OneDrive\Documentos\INGENIERIA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 ls
-la /usr/lib/python3/dist-packages/odoo/addons/
total 2600
drwxr-xr-x 644 root root 24576 Sep  9 02:38 .
drwxr-xr-x 12 root root 4096 Sep  9 02:38 ..
-rw-r--r--  1 root root 814 Sep  8 01:03 __init__.py
drwxr-xr-x  2 root root 4096 Sep  9 02:38 __pycache__
drwxr-xr-x 17 root root 4096 Sep  9 02:38 account
drwxr-xr-x  6 root root 4096 Sep  9 02:38 account_add_gln
drwxr-xr-x  8 root root 4096 Sep  9 02:38 account_audit_trail
drwxr-xr-x 11 root root 4096 Sep  9 02:38 account_check_printing
drwxr-xr-x  9 root root 4096 Sep  9 02:38 account_debit_note
drwxr-xr-x  6 root root 4096 Sep  9 02:38 account_debit_note_sequence
drwxr-xr-x 10 root root 4096 Sep  9 02:38 account_edi
drwxr-xr-x  9 root root 4096 Sep  9 02:38 account_edi_proxy_client
drwxr-xr-x  9 root root 4096 Sep  9 02:38 account_edi_ubl_cii
drwxr-xr-x  7 root root 4096 Sep  9 02:38 account_edi_ubl_cii_tax_extension
drwxr-xr-x  9 root root 4096 Sep  9 02:38 account_fleet
drwxr-xr-x  5 root root 4096 Sep  9 02:38 account_lock
drwxr-xr-x 11 root root 4096 Sep  9 02:38 account_payment
drwxr-xr-x  8 root root 4096 Sep  9 02:38 account_payment_term
drwxr-xr-x 12 root root 4096 Sep  9 02:38 account_peppol
drwxr-xr-x 10 root root 4096 Sep  9 02:38 account_peppol_response
drwxr-xr-x  8 root root 4096 Sep  9 02:38 account_peppol_selfbilling
drwxr-xr-x  6 root root 4096 Sep  9 02:38 account_qr_code_emv
drwxr-xr-x  6 root root 4096 Sep  9 02:38 account_qr_code_sepa
drwxr-xr-x  7 root root 4096 Sep  9 02:38 account_tax_python
drwxr-xr-x  9 root root 4096 Sep  9 02:38 account_test
drwxr-xr-x  8 root root 4096 Sep  9 02:38 account_update_tax_tags
drwxr-xr-x 12 root root 4096 Sep  9 02:38 analytic
drwxr-xr-x  6 root root 4096 Sep  9 02:38 attachment_indexation
drwxr-xr-x  8 root root 4096 Sep  9 02:38 auth_ldap
drwxr-xr-x 10 root root 4096 Sep  9 02:38 auth_oauth
drwxr-xr-x  8 root root 4096 Sep  9 02:38 auth_password_policy
drwxr-xr-x  5 root root 4096 Sep  9 02:38 auth_password_policy_portal
drwxr-xr-x  5 root root 4096 Sep  9 02:38 auth_password_policy_signup
drwxr-xr-x 10 root root 4096 Sep  9 02:38 auth_signup
drwxr-xr-x 12 root root 4096 Sep  9 02:38 auth_totp
drwxr-xr-x  9 root root 4096 Sep  9 02:38 auth_totp_mail
drwxr-xr-x 10 root root 4096 Sep  9 02:38 auth_totp_mail_enforce
```

3. Identificar módulos core vs módulos adicionales:

```
# Módulos base
```

```
docker exec -it <contenedor_odoo> ls /usr/lib/python3/dist-packages/odoo/addons/ | head -20
```

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 sh
-c "ls /usr/lib/python3/dist-packages/odoo/addons/ | head -20"
__init__.py
__pycache__
account
account_add_gln
account_audit_trail
account_check_printing
account_debit_note
account_debit_note_sequence
account_edi
account_edi_proxy_client
account_edi_ubl_cii
account_edi_ubl_cii_tax_extension
account_fleet
account_lock
account_payment
account_payment_term
account_peppol
account_peppol_response
account_peppol_selfbilling
account_qr_code_emv
```

Módulos personalizados (si existe volumen montado)
 docker exec -it <contenedor_odoo> ls /mnt/extra-addons/

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 ls
-la /mnt/extra-addons/
total 8
drwxr-xr-x 2 odoo root 4096 Sep  9 02:38 .
drwxr-xr-x 1 root root 4096 Sep  9 02:38 ..
```

Se verificó el directorio /mnt/extra-addons/ dentro del contenedor de Odoo. El directorio se encuentra disponible, pero actualmente no contiene módulos personalizados, debido a que el entorno Docker Compose utilizado para la actividad no tiene módulos adicionales montados.

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
odoo-docker-db-1    postgres:15          "docker-entrypoint.s..." db         18 minutes ago Up 17 minutes 5432/tcp
odoo-docker-odoo-1  odoo:17              "/entrypoint.sh odoo" odoo       18 minutes ago Up 17 minutes 0.0.0.0:8069->8069/tcp, [::]:8069->8069/tcp
```

Se comprobó mediante Docker Compose que los servicios de Odoo y PostgreSQL se encuentran activos. Odoo está ejecutándose en el puerto 8069 y PostgreSQL utiliza el puerto 5432.

Preguntas de Análisis:

1. ¿Cuántos módulos base identificó en el directorio addons?

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 sh
-c "find /usr/lib/python3/dist-packages/odoo/addons/ -mindepth 1 -maxdepth 1 -type d | wc -l"
642
```

Respuesta: Se identificaron 642 módulos/directorios en el directorio addons de Odoo.

2. ¿Qué servicios componen la arquitectura de Odoo en Docker Compose?

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
odoo-docker-db-1    postgres:15          "docker-entrypoint.s..." db         18 minutes ago Up 17 minutes 5432/tcp
odoo-docker-odoo-1  odoo:17              "/entrypoint.sh odoo" odoo       18 minutes ago Up 17 minutes 0.0.0.0:8069->8069/tcp, [::]:8069->8069/tcp
```

Respuesta: La arquitectura de Odoo en Docker Compose está compuesta por dos servicios principales: Odoo, que ejecuta la aplicación y su lógica de negocio, y PostgreSQL, que funciona como sistema gestor de base de datos para almacenar la información de Odoo. Ambos servicios se comunican mediante la red `odoo-docker_default`.

3. ¿Cómo se comunica Odoo con PostgreSQL? Describa la conexión.

```
C:\Users\mayor\OneDrive\Documentos\INGENIERIA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 env
| findstr /I "HOST USER PASSWORD"
HOSTNAME=cfc33f28b58
PASSWORD=odoo
HOST=db
USER=odoo
```

Respuesta: Odoo se comunica con PostgreSQL mediante la red interna de Docker Compose `odoo-docker_default`. Odoo utiliza `db` como nombre del host para localizar el servicio PostgreSQL y se conecta mediante el puerto 5432, utilizando las credenciales configuradas (`odoo / odoo`).

4.2 Actividad 2: Análisis de un Módulo

□ **Tiempo estimado: 10 minutos**

Objetivo: Analizar la estructura del módulo `sale` (Ventas) de Odoo.

Procedimiento:

4. Explorar el módulo `sale`:

```
docker exec -it <contenedor_odoo> ls -la /usr/lib/python3/dist-packages/odoo/addons/sale/
```

```
C:\Users\mayor\OneDrive\Documentos\INGENIERIA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 ls
-la /usr/lib/python3/dist-packages/odoo/addons/sale/
total 88
drwxr-xr-x 14 root root 4096 Sep  9 02:38 .
drwxr-xr-x 644 root root 24576 Sep  9 02:38 ..
-rw-r--r--  1 root root 3616 Sep  8 01:03 README.md
-rw-r--r--  1 root root 487 Sep  8 01:03 __init__.py
-rw-r--r--  1 root root 3180 Sep  8 01:03 __manifest__.py
drwxr-xr-x  2 root root 4096 Sep  9 02:38 __pycache__
drwxr-xr-x  3 root root 4096 Sep  9 02:38 controllers
drwxr-xr-x  2 root root 4096 Sep  9 02:38 data
drwxr-xr-x  2 root root 4096 Sep  9 02:38 i18n
drwxr-xr-x  3 root root 4096 Sep  9 02:38 models
drwxr-xr-x  3 root root 4096 Sep  9 02:38 populate
drwxr-xr-x  3 root root 4096 Sep  9 02:38 report
drwxr-xr-x  2 root root 4096 Sep  9 02:38 security
drwxr-xr-x  6 root root 4096 Sep  9 02:38 static
drwxr-xr-x  3 root root 4096 Sep  9 02:38 tests
drwxr-xr-x  2 root root 4096 Sep  9 02:38 views
drwxr-xr-x  3 root root 4096 Sep  9 02:38 wizard
```

Ver el manifest

```
docker exec -it <contenedor_odoo> cat /usr/lib/python3/dist-
packages/odoo/addons/sale/__manifest__.py
```

```

C:\Windows\System32\cmd.exe
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec odoo-docker-odoo-1 cat
/usr/lib/python3/dist-packages/odoo/addons/sale/__manifest__.py
# -*- coding: utf-8 -*-
# Part of Odoo. See LICENSE file for full copyright and licensing details.

{
    'name': 'Sales',
    'version': '1.2',
    'category': 'Sales/Sales',
    'summary': 'Sales internal machinery',
    'description': """
This module contains all the common features of Sales Management and eCommerce.
""",
    'depends': [
        'sales_team',
        'account_payment', # -> account, payment, portal
        'utm',
    ],
    'data': [
        'security/ir_model.access.csv',
        'security/res_groups.xml',
        'security/ir_rules.xml',

        'report/account_invoice_report_views.xml',
        'report/ir_actions_report_templates.xml',
        'report/ir_actions_report.xml',
        'report/sale_report_views.xml',

        'data/ir_cron.xml',
        'data/ir_sequence_data.xml',
        'data/mail_activity_type_data.xml',
        'data/mail_message_subtype_data.xml',
        'data/mail_template_data.xml',
        'data/ir_config_parameter.xml', # Needs mail_template_data
        'data/onboarding_data.xml',

        'wizard/account_accrued_orders_wizard_views.xml',
        'wizard/mass_cancel_orders_views.xml',
        'wizard/payment_link_wizard_views.xml',
    ],
}

```

```

'wizard/payment_link_wizard_views.xml',
'wizard/res_config_settings_views.xml',
'wizard/sale_make_invoice_advance_views.xml',
'wizard/sale_order_cancel_views.xml',
'wizard/sale_order_discount_views.xml',

# Define sale order views before their references
'views/sale_order_views.xml',

'views/account_views.xml',
'views/crm_team_views.xml',
'views/mail_activity_views.xml',
'views/mail_activity_plan_views.xml',
'views/payment_views.xml',
'views/product_document_views.xml',
'views/product_packaging_views.xml',
'views/product_views.xml',
'views/res_partner_views.xml',
'views/sale_order_line_views.xml',
'views/sale_portal_templates.xml',
'views/utm_campaign_views.xml',

'views/sale_menus.xml', # Last because referencing actions defined in previous files
],
'demo': [
    'data/product_demo.xml',
    'data/sale_demo.xml',
],
'installable': True,
'assets': {
    'web.assets_backend': [
        'sale/static/src/scss/sale_onboarding.scss',
        'sale/static/src/js/sale_progressbar_field.js',
        'sale/static/src/js/tours/sale.js',
        'sale/static/src/js/sale_product_field.js',
        'sale/static/src/xml/**/*',
    ],
    'web.assets_frontend': [
        'sale/static/src/scss/sale_portal.scss',
        'sale/static/src/js/sale_portal_sidebar.js',
        'sale/static/src/js/sale_portal_prepayment.js',
    ],
}

```

```

        'sale/static/src/xml/**/*',
    ],
    'web.assets_frontend': [
        'sale/static/src/scss/sale_portal.scss',
        'sale/static/src/js/sale_portal_sidebar.js',
        'sale/static/src/js/sale_portal_prepayment.js',
        'sale/static/src/js/sale_portal.js',
    ],
    'web.assets_tests': [
        'sale/static/tests/tours/**/*',
    ],
    'web.qunit_suite_tests': [
        'sale/static/tests/**/*',
        ('remove', 'sale/static/tests/tours/**/*')
    ],
    'web.report_assets_common': [
        'sale/static/src/scss/sale_report.scss',
    ],
},
'post_init_hook': '_synchronize_cron',
'license': 'LGPL-3',
}

```

5. Analizar un modelo:

docker exec -it <contenedor_odoo> head -100 /usr/lib/python3/dist-packages/odoo/addons/sale/models/sale_order.py

```

C:\Windows\System32\cmd.exe
C:\Users\mayor\OneDrive\Documents\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec -it odoo-docker-odoo-1
head -100 /usr/lib/python3/dist-packages/odoo/addons/sale/models/sale_order.py
# Part of Odoo. See LICENSE file for full copyright and licensing details.
from collections import defaultdict
from datetime import timedelta
from itertools import groupby

from odoo import api, fields, models, SUPERUSER_ID, _
from odoo.exceptions import AccessError, UserError, ValidationError
from odoo.fields import Command
from odoo.osv import expression
from odoo.tools import float_is_zero, format_amount, format_date, html_keep_url, is_html_empty
from odoo.tools.sql import create_index

from odoo.addons.payment import utils as payment_utils

INVOICE_STATUS = [
    ('upselling', 'Upselling Opportunity'),
    ('invoiced', 'Fully Invoiced'),
    ('to invoice', 'To Invoice'),
    ('no', 'Nothing to Invoice')
]

SALE_ORDER_STATE = [
    ('draft', 'Quotation'),
    ('sent', 'Quotation Sent'),
    ('sale', 'Sales Order'),
    ('cancel', 'Cancelled'),
]

class SaleOrder(models.Model):
    _name = 'sale.order'
    _inherit = ['portal.mixin', 'product.catalog.mixin', 'mail.thread', 'mail.activity.mixin', 'utm.mixin']
    _description = 'Sales Order'
    _order = 'date_order desc, id desc'
    _check_company_auto = True

    _sql_constraints = [
        ('date_order_conditional_required',

```

```

CHECK((state = 'sale' AND date_order IS NOT NULL) OR state != 'sale')),
"A confirmed sales order requires a confirmation date."),
]

@property
def _rec_names_search(self):
    if self._context.get('sale_show_partner_name'):
        return ['name', 'partner_id.name']
    return ['name']

#== FIELDS ==#

name = fields.Char(
    string="Order Reference",
    required=True, copy=False, readonly=False,
    index='trigram',
    default=lambda self: _('New'))

company_id = fields.Many2one(
    comodel_name='res.company',
    required=True, index=True,
    default=lambda self: self.env.company)
partner_id = fields.Many2one(
    comodel_name='res.partner',
    string="Customer",
    required=True, change_default=True, index=True,
    tracking=1,
    check_company=True)
state = fields.Selection(
    selection=SALE_ORDER_STATE,
    string="Status",
    readonly=True, copy=False, index=True,
    tracking=3,
    group_expand=True,
    default='draft')
locked = fields.Boolean(default=False, copy=False, help="Locked orders cannot be modified.")

client_order_ref = fields.Char(string="Customer Reference", copy=False)
create_date = fields.Datetime( # Override of default create_date field from ORM
    string="Creation Date", index=True, readonly=True)
commitment_date = fields.Datetime(

```

```

    string="Delivery Date", copy=False,
    help="This is the delivery date promised to the customer. "
    "If set, the delivery order will be scheduled based on "
    "this date rather than product lead times.")
date_order = fields.Datetime(
    string="Order Date",
    required=True, copy=False,
    help="Creation date of draft/sent orders,\nConfirmation date of confirmed orders.",
    default=fields.Datetime.now)
origin = fields.Char(
    string="Source Document",
    help="Reference of the document that generated this sales order request")
reference = fields.Char(
    string="Payment Ref.",
    help="The payment communication of this sale order.",
    copy=False)

require_signature = fields.Boolean(
    string="Online signature",
    compute='_compute_require_signature',
    store=True, readonly=False, precompute=True,

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug odoo-docker-odoo-1
Learn more at https://docs.docker.com/go/debug-cli/

```

El archivo `sale_order.py` define la clase `SaleOrder`, que hereda de `models.Model` y representa el modelo de pedidos de venta de Odoo. El nombre técnico del modelo es `sale.order`. El modelo utiliza el ORM de Odoo mediante `models.Model` y define diferentes campos, como el nombre del pedido, la empresa, el cliente, el estado y la fecha del pedido. También utiliza herencia de funcionalidades mediante `_inherit`, incorporando características relacionadas con portal, productos, comunicación y actividades.

6. Examinar las vistas:

```
docker exec -it <contenedor_odoo> ls /usr/lib/python3/dist-packages/odoo/addons/sale/views/
```

```

C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>docker exec -it odoo-docker-odoo-1
ls /usr/lib/python3/dist-packages/odoo/addons/sale/views/
account_views.xml      mail_activity_views.xml  product_packaging_views.xml  sale_menus.xml          sale_portal_templates.xml
crm_team_views.xml     payment_views.xml        product_views.xml           sale_order_line_views.xml  utm_campaign_views.xml
mail_activity_plan_views.xml  product_document_views.xml  res_partner_views.xml       sale_order_views.xml

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug odoo-docker-odoo-1
Learn more at https://docs.docker.com/go/debug-cli/

```

El módulo sale contiene diferentes archivos XML dentro de la carpeta views, encargados de definir la presentación de la información y los menús del módulo. Entre ellos se encuentran sale_order_views.xml, sale_order_line_views.xml, product_views.xml, payment_views.xml y sale_menus.xml. Estas vistas permiten representar y organizar la información de los modelos en la interfaz de Odoo.

Tabla de Análisis del Módulo:

Complete la siguiente tabla con la información del módulo sale:

Aspecto	Descripción
Nombre técnico	sale
Versión	1.2
Dependencias	sales_team, account_payment, utm
Modelos principales	sale.order, sale.order.line, además de modelos relacionados con productos, clientes, pagos y contabilidad.
Vistas XML	sale_order_views.xml, sale_order_line_views.xml, sale_menus.xml, product_views.xml, payment_views.xml, res_partner_views.xml, entre otras.
Archivos JS/OWL	sale_progressbar_field.js, sale_product_field.js, sale.js, sale_portal.js, sale_portal_sidebar.js, sale_portal_prepayment.js.

4.3 Actividad 3: Introducción a OWL

□ **Tiempo estimado: 20 minutos**

Objetivo: Crear un componente OWL básico y entender su ciclo de vida.

Conceptos Clave de OWL:

Concepto	Descripción
Component	Clase base para crear componentes. Contiene template, estado y métodos.
useState	Hook para crear estado reactivo. Cuando cambia, el componente se re-renderiza.
useRef	Hook para obtener referencia directa a elementos del DOM.
onMounted	Hook del ciclo de vida. Se ejecuta cuando el componente se monta en el DOM.

onWillUnmount	Hook del ciclo de vida. Se ejecuta antes de que el componente se desmonte.
props	Propiedades recibidas del componente padre. Son de solo lectura.

Ejemplo de Componente OWL:

Analice el siguiente componente contador:

```
/** @odoo-module */

import { Component, useState } from '@odoo/owl';

export class Counter extends Component {
    static template = 'mi_modulo.Counter';

    setup() {
        this.state = useState({ count: 0 });
    }

    increment() {
        this.state.count++;
    }

    decrement() {
        this.state.count--;
    }
}
```

Plantilla QWeb correspondiente (XML):

```
<?xml version="1.0" encoding="UTF-8"?>
<templates>
    <t t-name="mi_modulo.Counter">
        <div class="o_counter p-3 bg-light rounded">
            <h4>Contador OWL</h4>
            <div class="d-flex align-items-center gap-3">
                <button class="btn btn-danger" t-on-click="decrement">-</button>
                <span class="fs-2 fw-bold" t-esc="state.count"/>
                <button class="btn btn-success" t-on-click="increment">+</button>
            </div>
        </div>
    </t>
</templates>
```

Preguntas de Comprensión:

1. ¿Qué hace la directiva t-on-click en las plantillas QWeb?

Respuesta: La directiva t-on-click permite asociar una acción o método de JavaScript al evento de clic de un elemento. Por ejemplo, t-on-click="increment" hace que al presionar el botón se ejecute el método increment() del componente OWL.

2. ¿Por qué usamos useState en lugar de una variable normal?

Respuesta: Usamos `useState` porque permite crear un estado reactivo en el componente. Cuando el valor del estado cambia, OWL detecta el cambio y vuelve a renderizar el componente para actualizar la interfaz. Una variable normal no proporciona este mecanismo de re-renderizado automático.

3. ¿Qué significa `static template` y por qué es necesario?

Respuesta: `static template` indica el nombre de la plantilla QWeb que utilizará el componente OWL para construir y mostrar su interfaz. Es necesario porque permite relacionar la clase JavaScript del componente con la plantilla XML correspondiente. En el ejemplo, `static template = 'mi_modulo.Counter'`; se relaciona con la plantilla `<t t-name="mi_modulo.Counter">`

4.4 Actividad 4: Desarrollo Práctico

□ Tiempo estimado: 30 minutos

Objetivo: Crear un módulo personalizado con un componente OWL funcional.

Paso 1: Crear estructura del módulo

```
# Crear directorio del módulo en el volumen de addons
mkdir -p /mnt/extra-addons/mi_componente_owl
cd /mnt/extra-addons/mi_componente_owl
```

```
# Crear estructura
mkdir -p static/src/{js,xml,scss}
mkdir -p views
touch __init__.py
```

```
C:\USERS\MAYOR\ONEDRIVE\DOCUMENTOS\INGENIERIA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\ODOO-DOCKER\MI_COMPONENTE_OWL
__init__.py
__manifest__.py
static
├── src
│   ├── js
│   │   └── todo_list.js
│   ├── scss
│   │   └── todo_list.scss
│   └── xml
│       └── todo_list.xml
└── views
    └── todo_action.xml
__pycache__
    └── __init__.cpython-310.pyc
```

Paso 2: Crear __manifest__.py

```
{
    'name': 'Mi Componente OWL',
    'version': '17.0.1.0.0',
    'category': 'Tools',
    'summary': 'Laboratorio de aprendizaje OWL',
    'depends': ['web'],
    'assets': {
        'web.assets_backend': [
            'mi_componente_owl/static/src/js/**/*',
            'mi_componente_owl/static/src/xml/**/*',
            'mi_componente_owl/static/src/scss/**/*',
        ],
    },
    'installable': True,
    'application': False,
}
```

```
C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CO
RTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>type mi_componente_owl\__manif
est__.py
{
    'name': 'Mi Componente OWL',
    'version': '17.0.1.0.0',
    'category': 'Tools',
    'summary': 'Laboratorio de aprendizaje OWL',
    'depends': ['web'],
    'data': [
        'views/todo_action.xml',
    ],
    'assets': {
        'web.assets_backend': [
            'mi_componente_owl/static/src/js/**/*',
            'mi_componente_owl/static/src/xml/**/*',
            'mi_componente_owl/static/src/scss/**/*',
        ],
    },
    'installable': True,
    'application': False,
}
```

Contenido del archivo __manifest__.py, donde se configuran los datos, dependencias y recursos del módulo OWL.

Paso 3: Crear componente JS

Archivo: static/src/js/todo_list.js

```
/** @odoo-module */

import { Component, useState } from '@odoo/owl';
import { registry } from '@web/core/registry';

export class TodoList extends Component {
    static template = 'mi_componente_owl.TodoList';

    setup() {
```

```

        this.state = useState({
            tasks: [],
            newTask: ''
        });
    }

    addTask() {
        if (this.state.newTask.trim()) {
            this.state.tasks.push({
                id: Date.now(),
                text: this.state.newTask,
                done: false
            });
            this.state.newTask = '';
        }
    }

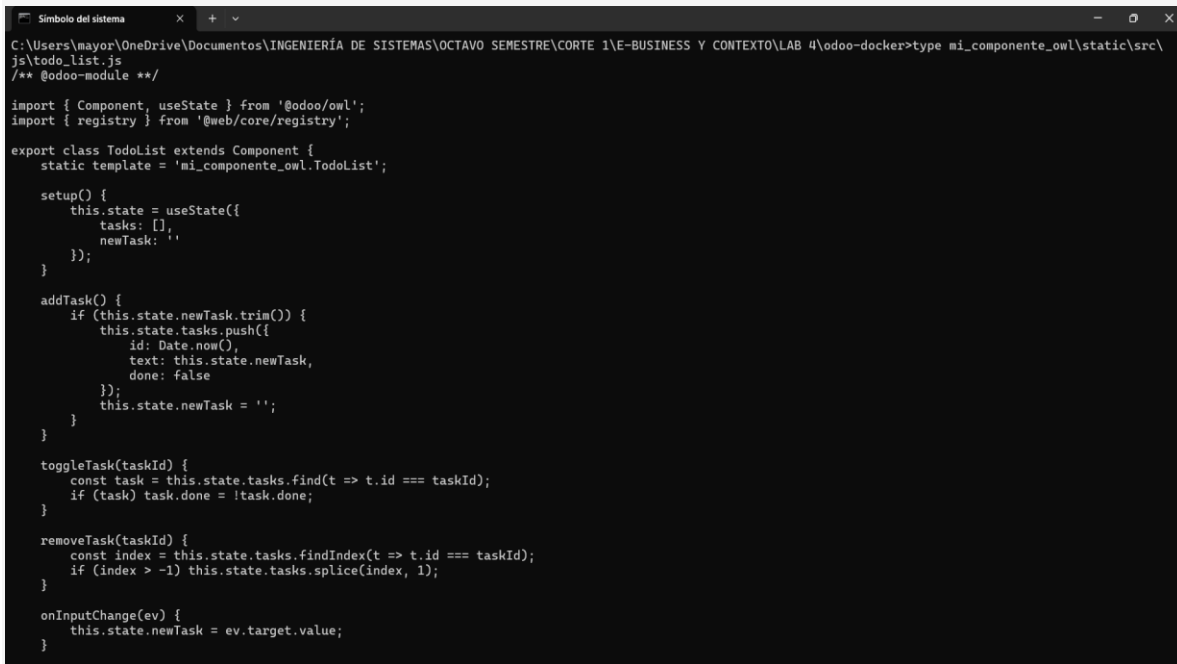
    toggleTask(taskId) {
        const task = this.state.tasks.find(t => t.id === taskId);
        if (task) task.done = !task.done;
    }

    removeTask(taskId) {
        const index = this.state.tasks.findIndex(t => t.id === taskId);
        if (index > -1) this.state.tasks.splice(index, 1);
    }

    onInputChange(ev) {
        this.state.newTask = ev.target.value;
    }
}

registry.category('actions').add('mi_componente_owl.todo_action', TodoList);

```



```

C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>type mi_componente_owl\static\src\js\todo_list.js
/** @odoo-module **/

import { Component, useState } from '@odoo/owl';
import { registry } from '@web/core/registry';

export class TodoList extends Component {
    static template = 'mi_componente_owl.TodoList';

    setup() {
        this.state = useState({
            tasks: [],
            newTask: ''
        });
    }

    addTask() {
        if (this.state.newTask.trim()) {
            this.state.tasks.push({
                id: Date.now(),
                text: this.state.newTask,
                done: false
            });
            this.state.newTask = '';
        }
    }

    toggleTask(taskId) {
        const task = this.state.tasks.find(t => t.id === taskId);
        if (task) task.done = !task.done;
    }

    removeTask(taskId) {
        const index = this.state.tasks.findIndex(t => t.id === taskId);
        if (index > -1) this.state.tasks.splice(index, 1);
    }

    onInputChange(ev) {
        this.state.newTask = ev.target.value;
    }
}

```

```

    onKeyUp(ev) {
        if (ev.key === 'Enter') {
            this.addTask();
        }
    }
}

```

Código JavaScript del componente TodoList, encargado de gestionar las tareas y la interacción del componente OWL.

Paso 4: Crear plantilla XML

Archivo: *static/src/xml/todo_list.xml*

```

<?xml version="1.0" encoding="UTF-8"?>
<templates>
    <t t-name="mi_componente_owl.TodoList">
        <div class="o_todo_list p-4">
            <h2 class="mb-4">Lista de Tareas (OWL)</h2>

            <div class="input-group mb-3">
                <input type="text" class="form-control"
                    placeholder="Nueva tarea..."
                    t-att-value="state.newTask"
                    t-on-input="onInputChange"
                    t-on-keyup.enter="addTask"/>
                <button class="btn btn-primary" t-on-click="addTask">
                    Agregar
                </button>
            </div>

            <ul class="list-group">
                <t t-foreach="state.tasks" t-as="task" t-key="task.id">
                    <li class="list-group-item d-flex justify-content-between">
                        <div class="form-check">
                            <input type="checkbox" class="form-check-input"
                                t-att-checked="task.done"
                                t-on-change="() => this.toggleTask(task.id)"/>
                            <label t-att-class="task.done ? 'text-decoration-line-through
text-muted' : ''"
                                t-esc="task.text"/>
                        </div>
                        <button class="btn btn-sm btn-outline-danger"
                            t-on-click="() => this.removeTask(task.id)">
                            X
                        </button>
                    </li>
                </t>
            </ul>

            <p class="mt-3 text-muted" t-if="state.tasks.length === 0">
                No hay tareas. ¡Agrega una!
            </p>
        </div>
    </t>

```

</templates>

```

C:\Users\mayor\OneDrive\Documentos\INGENIERÍA DE SISTEMAS\OCTAVO SEMESTRE\CORTE 1\E-BUSINESS Y CONTEXTO\LAB 4\odoo-docker>type mi_componente_owl\static\src\
xml\todo_list.xml
<?xml version="1.0" encoding="UTF-8"?>
<templates>
  <t t-name="mi_componente_owl.TodoList">
    <div class="o_todo_list p-4">
      <h2 class="mb-4">Lista de Tareas (OWL)</h2>

      <div class="input-group mb-3">
        <input type="text" class="form-control"
          placeholder="Nueva tarea..."
          t-att-value="state.newTask"
          t-on-input="onInputChange"
          t-on-keyup="onKeyUp" />
        <button class="btn btn-primary" t-on-click="addTask">
          Agregar
        </button>
      </div>

      <ul class="list-group">
        <t t-foreach="state.tasks" t-as="task" t-key="task.id">
          <li class="list-group-item d-flex justify-content-between">
            <div class="form-check">
              <input type="checkbox" class="form-check-input"
                t-att-checked="task.done"
                t-on-change="() => this.toggleTask(task.id)" />
              <label t-att-class="task.done ? 'text-decoration-line-through text-muted' : ''"
                t-esc="task.text" />
            </div>
            <button class="btn btn-sm btn-outline-danger"
              t-on-click="() => this.removeTask(task.id)">
              ðŸšŒ
            </button>
          </li>
        </t>
      </ul>

      <p class="mt-3 text-muted" t-if="state.tasks.length === 0">
        No hay tareas. ¡Agrega una!
      </p>
    </div>
  </t>
</templates>

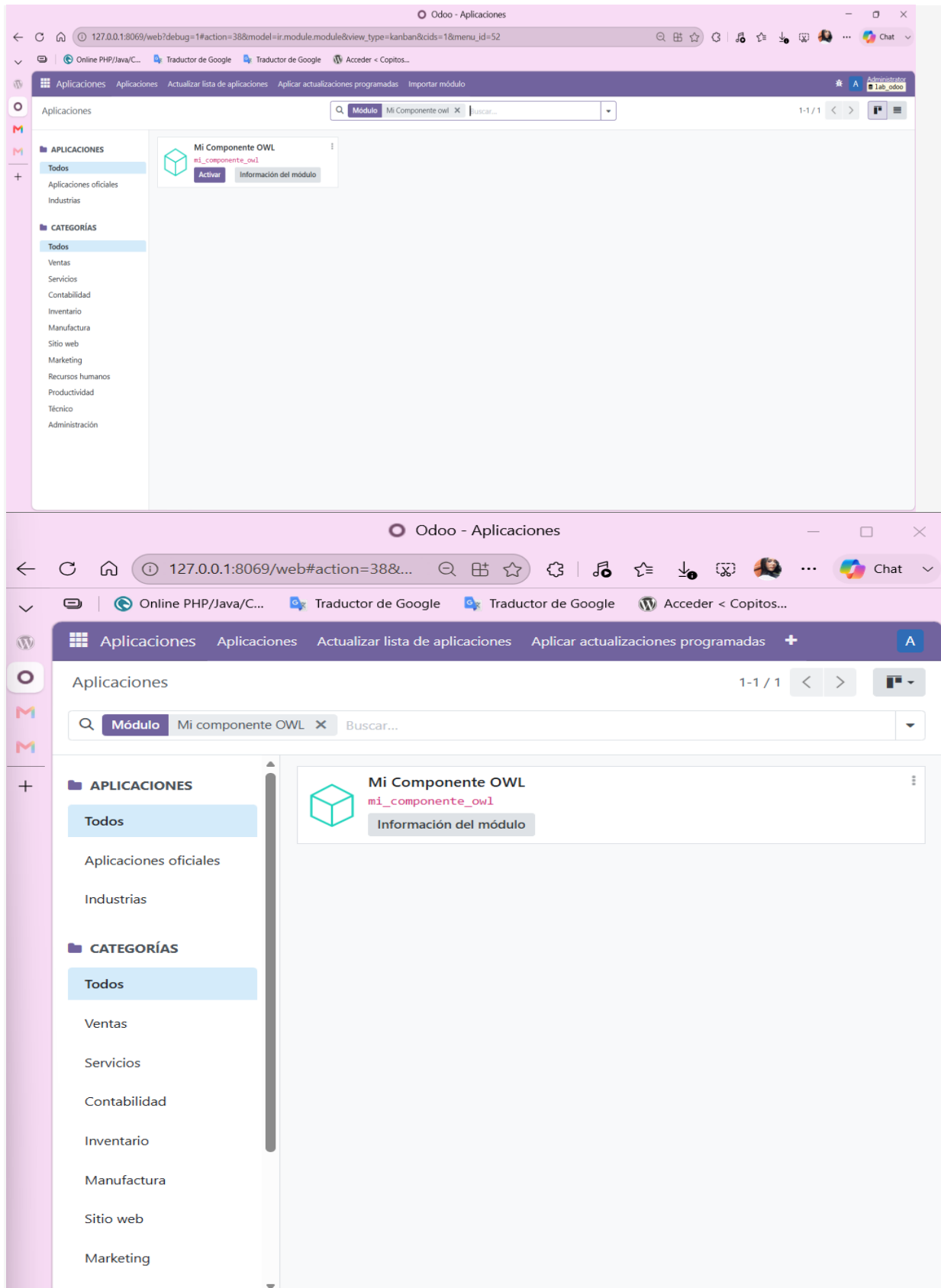
```

Plantilla XML/QWeb del componente TodoList, donde se define la interfaz y las interacciones del componente OWL.

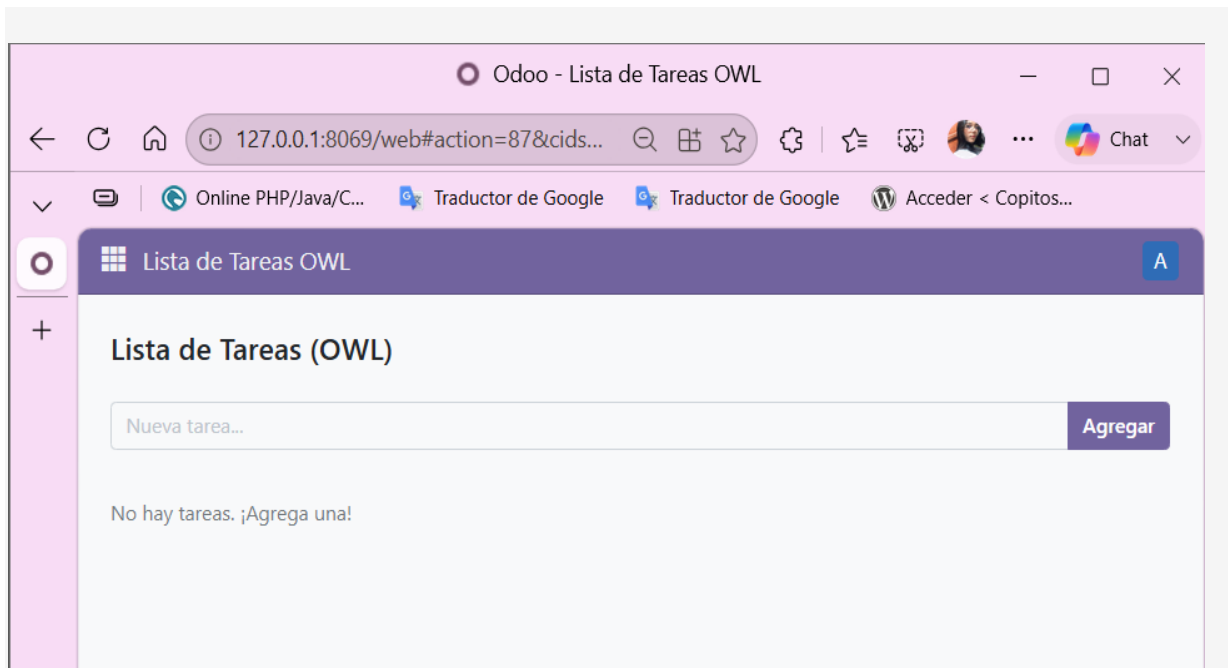
Paso 5: Instalar y probar

```
# Reiniciar Odoo para cargar el módulo
docker restart <contenedor_odoo>
```

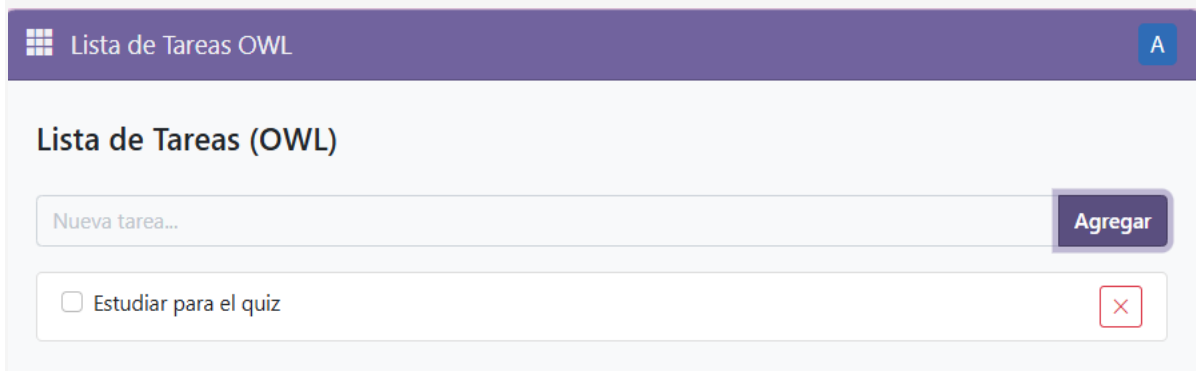
```
# En Odoo: Aplicaciones > Actualizar lista de aplicaciones
# Buscar e instalar 'Mi Componente OWL'
```



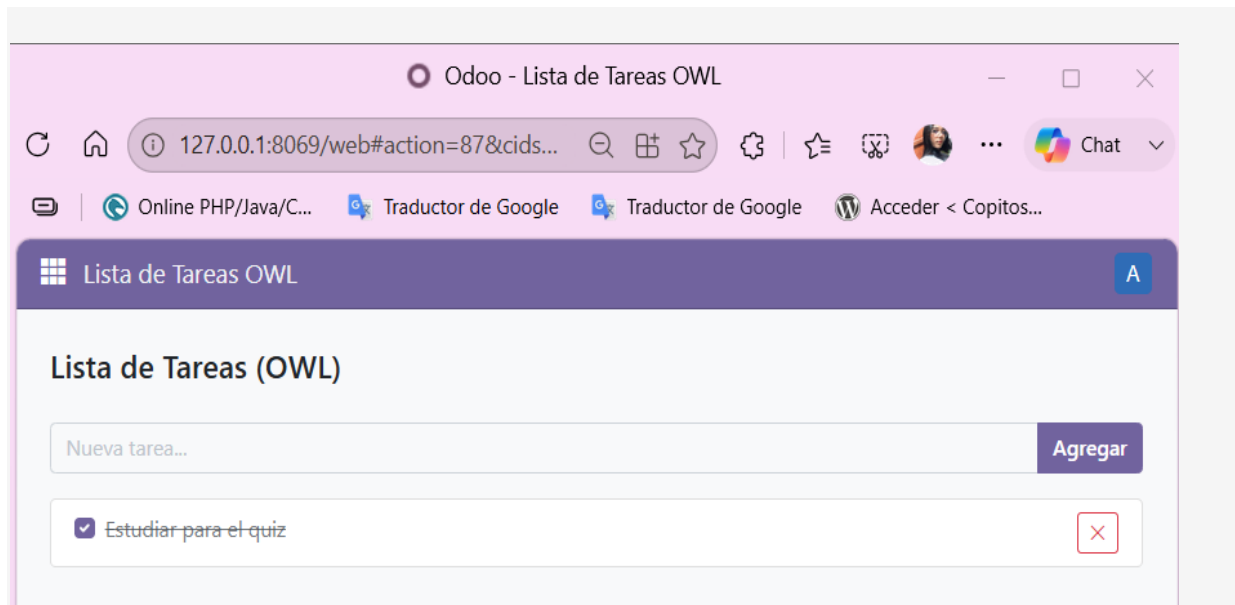
Evidencia: Módulo “Mi Componente OWL” instalado correctamente en Odoo.



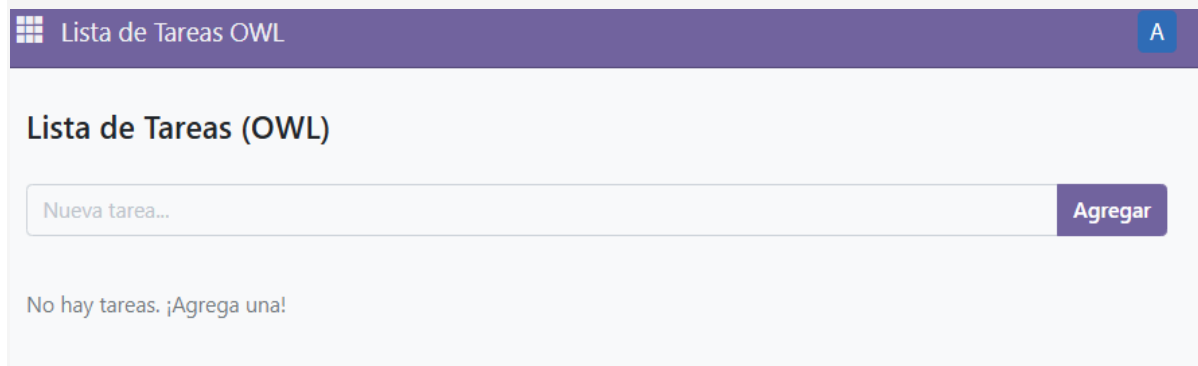
Componente “Lista de Tareas OWL” ejecutándose correctamente en Odoo 17.



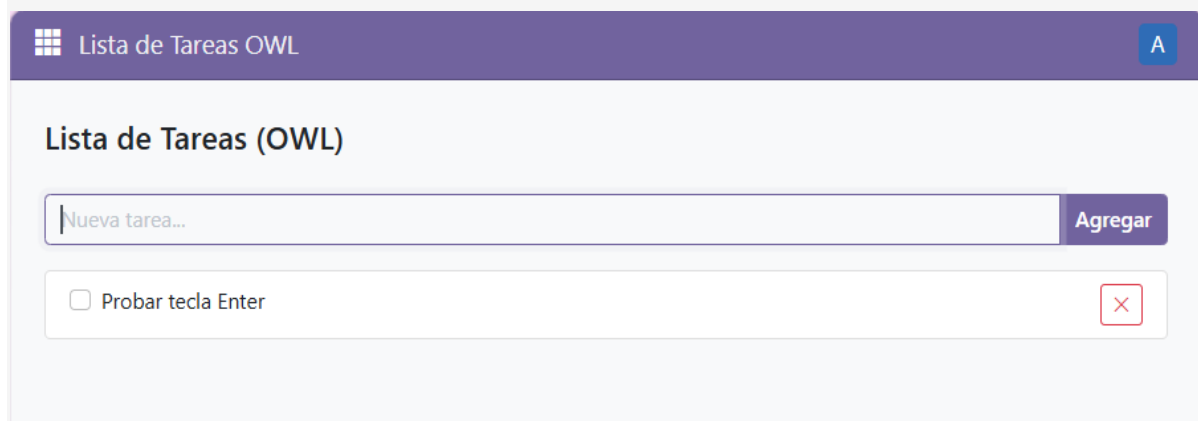
Componente OWL funcionando y mostrando una tarea agregada dinámicamente.



Tarea completada.



Tarea eliminada.



Tarea agregada con la tecla Enter.

5. Entregables

#	Entregable	Formato	Peso
1	Guía de laboratorio completada con respuestas	PDF/Word	30%
2	Módulo OWL funcional (código fuente)	ZIP	40%
3	Imágenes del componente ()	Jpg-png	20%
4	Diagrama de arquitectura de Odoo	PNG/PDF	10%

6. Rúbrica de Evaluación

Criterio	Excelente (5.0)	Bueno (4.0)	Por mejorar (3.0)
Comprensión arquitectura	Identifica y explica las 3 capas con precisión técnica	Identifica las capas pero con explicaciones básicas	Confunde o mezcla responsabilidades de las capas
Componente OWL	Componente funcional con estado, eventos y estilos	Componente funcional con estado básico	Componente incompleto o con errores
Integración Odoo	Módulo instalable, registrado correctamente	Módulo con pequeños ajustes necesarios	Módulo no instalable o con errores críticos
Documentación	Respuestas completas, código comentado	Respuestas correctas, código parcialmente comentado	Respuestas incompletas, sin comentarios

7. Recursos Adicionales

7.1 Documentación Oficial

- Odoo Documentation: <https://www.odoo.com/documentation/17.0/>
- OWL Framework: <https://github.com/odoo/owl>
- Odoo Developer Tutorials: <https://www.odoo.com/documentation/17.0/developer/tutorials.html>

7.2 Comandos Docker Útiles

```
# Ver logs en tiempo real
docker logs -f <contenedor_odoo>
```

```
# Reiniciar Odoo con actualización de módulo
docker exec <contenedor_odoo> odoo -u mi_modulo -d <base_datos>
```

```
# Acceder a PostgreSQL  
docker exec -it <contenedor_postgres> psql -U odoo
```

8. Notas del Estudiante

Espacio para Reflexión durante el laboratorio: